Recursion

Recursion: A Self-Referential Approach

Recursion is a concept to solve a problem depends on solutions to smaller instances of the **exact same problem**.

How to form a recursive function :

1. The Recursive Status

This is the parameters of the function that is important to me to evaluate the solution

2. The Recursive Transition

This is where the function calls itself, but with a **modified Argument** that moves the problem closer to the base case.

3. The Base Case (The Stopper)

The base case is a termination condition. When this condition is met, the function stops calling itself and returns a result.

> !! **Note:** Without a base case, the function runs forever, leading to a stack overflow error.

Example: The Factorial Function

The factorial of a non-negative integer n (written as $n!$) is the product of all positive integers less than or equal to n.

The formal, iterative definition is:

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 1$$

The recursive definition is much cleaner:

$$n! = n \times (n-1)!$$

Breakdown of `Factorial(4)`

Here is how the calculation breaks down into the base case and the recursive step:

Function Call	Action	Return Value
`Factorial(4)`	$4 \times \text{Factorial}(3)$	$\mathbf{24}$
`Factorial(3)`	$3 \times \text{Factorial}(2)$	6
`Factorial(2)`	$2 \times \text{Factorial}(1)$	2
`Factorial(1)`	**Base Case:** Return 1	1

The function unwinds: $4 \times (3 \times (2 \times 1)) = 24$.

Recursion vs. Iteration

Any problem solved by recursion can also be solved by iteration (using loops), and vice-versa *?*.

Feature	Recursion	Iteration (Loops)
Clarity	Often more elegant for specific problems (e.g., tree traversal).	Could be hard for deep or complex nested logic.
Execution	Less efficient due to the overhead of creating many function calls on the **call stack**.	Generally more efficient and faster, with less memory overhead.
Risk	High risk of **Stack Overflow** errors if the base case is missed or unreachable.	No risk of stack overflow; prone to **Off-by-One** errors in loop conditions.

When to use Recursion

Recursion is best used when:

1. The problem definition is inherently recursive (like the factorial).
2. Working with data structures defined recursively (like trees and graphs).
3. The recursive solution is significantly simpler and clearer than the iterative one (e.g., algorithms like ***Quick Sort*** or ***Merge Sort***).

Problems for Today

- Problem 1 : https://cses.fi/problemset/result/15498169/
- Solution

```
void print(int i) {
    if (i == 1) {
        cout << 1 << " ";
        return;
    }
    cout << i << " ";
    print((i%2 ? (i*3)+1 : i/2));
}
```

- Problem 2 : https://codeforces.com/group/MWSDmqGsZm/contest/223339/problem/V
- Solution

```
#include <bits/stdc++.h>
using namespace std;

int n , t , a[22];
bool rec(int i , int sum) {
    if (i == n) return (sum == t);

    return rec(i+1 , sum+a[i]) or rec(i+1 , sum-a[i]);
}



int main(){
    cin >> n >> t;

    for (int i = 0 ;i<n ;i++) cin >> a[i];

    cout << (rec(1 , a[0]) ? "YES":"NO");
}
```

- Problem 3 : https://codeforces.com/group/MWSDmqGsZm/contest/223339/problem/U
- Solution

```
#include <bits/stdc++.h>
using namespace std;

int n , w , weight[22] , value[22];

int rec(int i , int rem_w) {

    if(i==n) return 0;

    int ch1 = 0;
    if(rem_w >= weight[i]) ch1 = rec(i+1 , rem_w-weight[i]) + value[i];
    int ch2 = rec(i+1 , rem_w);
    return max(ch1 , ch2);
}



int main(){

    cin >> n >> w;
    for (int i = 0 ;i<n ;i++) cin >> weight[i] >> value[i];
    cout << rec(0 , w) << endl;

    return 0;
}
```